

درس الفصل الثالث عشر

# الأشجار الثنائية

العقد، الجذر والأوراق، طرق المرور، وشجرة البحث الثنائية


**الأستاذ:** د. بن عبد الرزاق يوسف

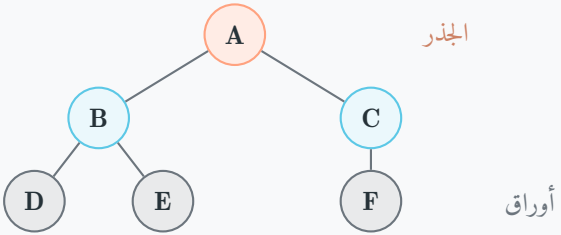
**الفئة:** طلبة السنة الأولى

# أهداف الدرس

في نهاية هذا الدرس، تستطيع أن:

- تعرّف الشجرة الثنائية ومفرداتها: الجذر، الابن، الورقة، الارتفاع.
- تمر على شجرة بالطرق الثلاث باستعمال دوال عودية.
- تضيف قيمة إلى شجرة بحث ثنائية وتبحث عنها.

# الشجرة




**الشجرة**: بنية هرمية تبدأ **بجذر**، ولكل عقدة **أبناء**. العقدة دون أبناء **ورقة**.

# مفردات الشجرة

| الكلمة | المعنى |
|---|---|
| الجذر | العقدة الأولى في الأعلى، ليس لها أب |
| الابن والأب | العقدة B ابن للعقدة A |
| الورقة | عقدة ليس لها أي ابن |
| الارتفاع | عدد المستويات من الجذر إلى أبعد ورقة |

# الشجرة الثنائية

**تعريف**

**الشجرة الثنائية:** شجرة لكل عقدة فيها ابنان على الأكثر: ابن أيسر وابن أيمن.

# تعريف العقدة

```c
typedef struct Node {
    int value;
    struct Node *left;     /* left child  */
    struct Node *right;    /* right child */
} Node;
```

مؤشران بدل مؤشر واحد في القائمة المتصلة. NULL يعني: لا يوجد ابن.

# إنشاء عقدة

```
Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = NULL;
    n->right = NULL;
    return n;
}
```

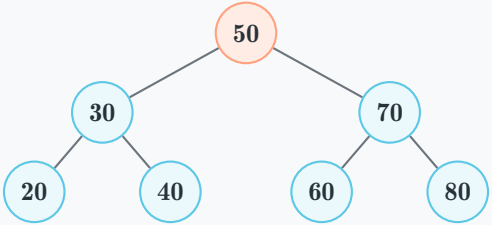
العقدة الجديدة ورقة: ليس لها أبناء.

# بناء شجرة صغيرة

```
Node *r = newNode(50);          /* root */
r->left = newNode(30);
r->right = newNode(70);
r->left->left = newNode(20);
r->left->right = newNode(40);
```

`r->left->left`: الابن الأيسر للابن الأيسر للجذر.

# شجرة البحث الثنائية




كل القيم في اليسار **أصغر** من العقدة، وكل القيم في اليمين **أكبر** منها.

# قاعدة شجرة البحث

**تعريف**

في **شجرة البحث الثنائية**: كل قيمة أصغر من العقدة توضع في **اليسار**، وكل قيمة أكبر منها توضع في **اليمين**.

# طرق المرور على الشجرة السابقة

| الطريقة | الترتيب | النتيجة |
|---|---|---|
| قبلي (preorder) | العقدة، اليسار، اليمين | 50 30 20 40 70 60 80 |
| وسطي (inorder) | اليسار، العقدة، اليمين | 20 30 40 50 60 70 80 |
| بعدي (postorder) | اليسار، اليمين، العقدة | 20 40 30 60 80 70 50 |

المرور الوسطي على شجرة بحث يعطي القيم مرتبة.

# المرور الوسطي

```c
void inorder(Node *t) {
    if (t == NULL)
        return;
    inorder(t->left);
    printf("%d ", t->value);
    inorder(t->right);
}
```

لتحويله إلى قبلي أو بعدي، نغيّر مكان `printf` فقط.

# الإضافة في شجرة البحث

```
Node *insert(Node *t, int v) {
    if (t == NULL)
        return newNode(v);
    if (v < t->value)
        t->left = insert(t->left, v);
    else
        t->right = insert(t->right, v);
    return t;
}
```

ننزل يسارًا أو يمينًا حتى نجد مكانًا فارغًا.

# برنامج كامل: إضافة ثم مرور

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *insert(Node *t, int v) {
    if (t == NULL) {
        t = malloc(sizeof(Node));
        t->value = v;
        t->left = t->right = NULL;
    } else if (v < t->value)
        t->left = insert(t->left, v);
    else
        t->right = insert(t->right, v);
    return t;
}
```

```c
void inorder(Node *t) {
    if (t == NULL) return;
    inorder(t->left);
    printf("%d ", t->value);
    inorder(t->right);
}

int main(void) {
    int v[7] = {50, 30, 70, 20, 40, 60, 80};
    int i;
    Node *root = NULL;
    for (i = 0; i < 7; i++)
        root = insert(root, v[i]);
    inorder(root);
    printf("\n");
    return 0;
}
```

```
20 30 40 50 60 70 80
```

# البحث في شجرة البحث

```c
int search(Node *t, int x) {
    if (t == NULL) return 0;          /* not found */
    if (x == t->value) return 1;      /* found */
    if (x < t->value)
        return search(t->left, x);
    return search(t->right, x);
}
```

في كل خطوة نترك نصف الشجرة: البحث سريع.

# عدد العقد والإرتفاع

```
1  int count(Node *t) {
2      if (t == NULL) return 0;
3      return 1 + count(t->left) + count(t->right);
4  }
5
6  int height(Node *t) {
7      int a, b;
8      if (t == NULL) return 0;
9      a = height(t->left);
10     b = height(t->right);
11     if (a > b) return 1 + a;
12     return 1 + b;
13 }
```

# تحرير الشجرة

```c
void freeTree(Node *t) {
    if (t == NULL)
        return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);          /* the node after its children */
}
```

نحرر الأبناء قبل العقدة: مرور بعدي.

# أمثلة تطبيقية

## عشرة أمثلة من مجالات مختلفة

نطبق ما تعلمناه في مجالات من الحياة اليومية: الصحة، الرياضة، الطقس، التجارة الإلكترونية، ألعاب الفيديو، الذكاء الاصطناعي، الشبكات الاجتماعية، نظام الملفات، البيئة، الهيكل التنظيمي.

# المثال الأول: الصحة، ترتيب الإستعجال

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
void inorder(Node *t) {
    if (t == NULL) return;
    inorder(t->left);
    printf("%d ", t->value);
    inorder(t->right);
}
int main(void) {
    Node *r = newNode(50);
    r->left = newNode(30);
    r->right = newNode(70);
    r->left->left = newNode(20);
    r->left->right = newNode(40);
    inorder(r);
    printf("\n");
    freeTree(r);
    return 0;
}
```

```
20 30 40 50 70
```

# المثال الثاني: الرياضة، أفضل نتيجة

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
int maxValue(Node *t) {
    while (t->right != NULL)
        t = t->right;
    return t->value;
}
int main(void) {
    Node *r = newNode(65);
    r->left = newNode(40);
    r->right = newNode(90);
    r->left->left = newNode(25);
    r->left->right = newNode(52);
    printf("Best = %d\n", maxValue(r));
    freeTree(r);
    return 0;
}
```

```
Best = 90
```

# المثال الثالث: الطقس، أدنى درجة حرارة

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
int minValue(Node *t) {
    while (t->left != NULL)
        t = t->left;
    return t->value;
}
int main(void) {
    Node *r = newNode(24);
    r->left = newNode(15);
    r->right = newNode(33);
    r->left->left = newNode(8);
    r->left->right = newNode(19);
    printf("Min = %d C\n", minValue(r));
    freeTree(r);
    return 0;
}
```

```
Min = 8 C
```

# المثال الرابع: التجارة الإلكترونية، مجموع الأسعار

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
int sum(Node *t) {
    if (t == NULL) return 0;
    return t->value + sum(t->left)
                    + sum(t->right);
}
int main(void) {
    Node *r = newNode(3000);
    r->left = newNode(1500);
    r->right = newNode(4500);
    r->left->left = newNode(800);
    r->left->right = newNode(2000);
    printf("Total = %d DA\n", sum(r));
    freeTree(r);
    return 0;
}
```

```
Total = 11800 DA
```

# المثال الخامس: ألعاب الفيديو، عمق خريطة المستويات

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
int height(Node *t) {
    if (t == NULL) return 0;
    int a = height(t->left);
    int b = height(t->right);
    if (a > b) return 1 + a;
    return 1 + b;
}
int main(void) {
    Node *r = newNode(5);
    r->left = newNode(3);
    r->right = newNode(8);
    r->left->left = newNode(1);
    r->left->right = newNode(4);
    printf("Height = %d\n", height(r));
    freeTree(r);
    return 0;
}
```

```
Height = 3
```

# المثال السادس: الذكاء الإصطناعي، شجرة القرار

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
int leaves(Node *t) {
    if (t == NULL) return 0;
    if (t->left == NULL && t->right == NULL)
        return 1;
    return leaves(t->left)
         + leaves(t->right);
}
int main(void) {
    Node *r = newNode(50);
    r->left = newNode(30);
    r->right = newNode(70);
    r->left->left = newNode(20);
    r->left->right = newNode(40);
    printf("Decisions = %d\n", leaves(r));
    freeTree(r);
    return 0;
}
```

```
Decisions = 3
```

# المثال السابع: الشبكات الإجتماعية، البحث عن مستخدم

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
int found(Node *t, int x) {
    if (t == NULL) return 0;
    if (x == t->value) return 1;
    if (x < t->value)
        return found(t->left, x);
    return found(t->right, x);
}
int main(void) {
    Node *r = newNode(1500);
    r->left = newNode(900);
    r->right = newNode(2400);
    r->left->left = newNode(400);
    r->left->right = newNode(1200);
    printf("User 1200: %d\n", found(r, 1200));
    freeTree(r);
    return 0;
}
```

```
User 1200: 1
```

# المثال الثامن: نظام الملفات، عدد المجلدات

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
int count(Node *t) {
    if (t == NULL) return 0;
    return 1 + count(t->left)
             + count(t->right);
}
int main(void) {
    Node *r = newNode(10);
    r->left = newNode(5);
    r->right = newNode(15);
    r->left->left = newNode(2);
    r->left->right = newNode(7);
    printf("Folders = %d\n", count(r));
    freeTree(r);
    return 0;
}
```

```
Folders = 5
```

# المثال التاسع: البيئة، قياسات ملوثة

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
int above(Node *t, int x) {
    if (t == NULL) return 0;
    int n = above(t->left, x)
          + above(t->right, x);
    if (t->value > x) n++;
    return n;
}
int main(void) {
    Node *r = newNode(110);
    r->left = newNode(60);
    r->right = newNode(150);
    r->left->left = newNode(40);
    r->left->right = newNode(105);
    printf("Polluted = %d\n", above(r, 100));
    freeTree(r);
    return 0;
}
```

```
Polluted = 3
```

# المثال العاشر: الهيكل التنظيمي، المدير قبل الموظفين

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *left, *right;
} Node;

Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void freeTree(Node *t) {
    if (t == NULL) return;
    freeTree(t->left);
    freeTree(t->right);
    free(t);
}
```

```c
void preorder(Node *t) {
    if (t == NULL) return;
    printf("%d ", t->value);
    preorder(t->left);
    preorder(t->right);
}
int main(void) {
    Node *r = newNode(100);
    r->left = newNode(50);
    r->right = newNode(150);
    r->left->left = newNode(20);
    r->left->right = newNode(70);
    preorder(r);
    printf("\n");
    freeTree(r);
    return 0;
}
```

```
100 50 20 70 150
```

# أخطاء شائعة

**تجنّب هذه الأخطاء**

- نسيان حالة التوقف `t == NULL` في الدوال العودية.
- وضع قيمة كبيرة في اليسار: لا تبقى الشجرة شجرة بحث.
- تحرير العقدة قبل أبنائها في `freeTree`.

# خلاصة الدرس

## خلاصة الدرس

- الشجرة الثنائية: لكل عقدة ابن أيسر وابن أيمن على الأكثر.
- نعالج الأشجار بدوال عودية: قبلي، وسطي، بعدي.
- في شجرة البحث: الأصغر يسارًا والأكبر يمينًا، والمرور الوسطي يعطي ترتيبًا تصاعديًا.